



Universiteti Metropolitan Tirana

Advanced Programming (JavaScript)

TASK MANAGER

Technical Project Report

Author

Blevis Allushi

Year 2, Semester 2 • Advanced Programming (JavaScript)

May 14, 2026

A full-stack task management application built using Node.js, Express.js, and a browser-based frontend.

Contents

1	Project Overview	2
2	Technology Stack	2
3	Project Structure	2
4	File-by-File Reference	3
4.1	server.js	3
4.2	app.js (root)	3
4.3	src/models/Task.js	4
4.4	src/data/store.js	5
4.5	src/middleware/validate.js	6
4.6	src/middleware/upload.js	7
4.7	src/routes/tasks.js	7
4.8	public/index.html	8
4.9	public/app.js	8
4.10	public/style.css	10
4.11	package.json	11
4.12	package-lock.json	11
4.13	test.http	11
4.14	.gitignore	12
4.15	.gitattributes	12
5	API Summary	13
6	Design Notes and Limitations	13

Project Overview

Overview

Task Manager is a full-stack web application for managing personal or team tasks. It consists of a RESTful API backend built with Node.js and Express.js, served alongside a static browser-based frontend. The two layers communicate entirely over HTTP using JSON, with no page reloads required.

Users can create tasks with a title, description, and priority level. Each task can be marked as complete, edited, deleted, and optionally have a single PDF file attached to it. A live statistics panel at the top of the page shows the total, completed, and pending task counts at all times.

All task data is held in server memory. No database is used, so data does not persist across server restarts. This was an intentional design decision to keep the project self-contained and dependency-free beyond its two npm packages.

Technology Stack

Concern	Technology	Version
Runtime	Node.js	≥ 18
Web framework	Express.js	5.2.1
File upload	Multer	2.1.1
Package manager	npm	bundled
Frontend	Vanilla HTML, CSS, JavaScript	—
Module system	CommonJS	—

Project Structure

```
task-manager/  
|-- public/  
| |-- index.html  
| |-- app.js  
| '-- style.css  
|-- src/  
| |-- data/  
| | '-- store.js  
| |-- middleware/  
| | |-- upload.js  
| | '-- validate.js  
| |-- models/  
| | '-- Task.js  
| '-- routes/  
| '-- tasks.js  
|-- uploads/  
|-- app.js  
|-- server.js  
|-- package.json  
|-- package-lock.json  
'-- test.http
```

The project is split into two clear zones. The

`public/`

directory holds everything that runs in the browser. The

`src/`

directory holds everything that runs on the server. The two root-level files

`server.js`

and

`app.js`

act as the server entry point and Express configuration respectively.

File-by-File Reference

`server.js`

This is the application entry point. It imports the configured Express application from

`app.js`

and calls

`listen`

on port 3000. It has no other responsibilities. Keeping this file separate from the application configuration means the Express app object can be imported and tested independently without binding a network port.

`app.js` (root)

Application setup

This file creates and exports the Express application instance. It performs the following steps in order:

1. Checks whether the

`uploads/`

directory exists on disk and creates it if not, using Node's built-in

`fs`

module. This prevents Multer from throwing an error on first run.

2. Registers

`express.json()`

middleware so that incoming request bodies with a

`Content-Type`

of

`application/json`

are automatically parsed into

```
req.body
```

```
.
```

3. Registers

```
express.static()
```

pointing at the

```
public/
```

directory, so the browser client files are served automatically.

4. Explicitly maps the root path

```
/
```

to serve

```
index.html
```

, ensuring the UI loads when a user visits the server in a browser.

5. Mounts the task router at

```
/tasks
```

```
.
```

6. Registers a catch-all 404 handler that returns a structured JSON error for any request that did not match a defined route.

src/models/Task.js

Defines the

```
Task
```

class, which is the data model for the entire application. Every task created by the API is an instance of this class.

The constructor accepts a title, description, and an optional priority (defaulting to "medium"). It assigns a numeric ID using

```
Date.now()
```

, a creation timestamp in ISO 8601 format, and sets

```
completed
```

to

```
false
```

```
.
```

The class exposes three methods:

- ```
complete()
```

— sets the

```
completed
```

property to

```
true
```

. This operation is one-way; there is no corresponding

```
uncomplete()
```

method.

- `update(fields)`

— accepts an object of fields to update. Only

```
title
```

,

```
description
```

, and

```
priority
```

are modifiable through this method. Any other keys in the input object are silently ignored.

- `toJSON()`

— returns a shallow copy of the instance as a plain object. Express calls this automatically when serialising a response with

```
res.json()
```

.

## `src/data/store.js`

This file is the in-memory data store. It holds a single private

```
tasks
```

array and exports six functions that provide the only interface through which other modules may read or modify that array.

| Function                                           | Description                                                                                                                      |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>getAllTasks()</code>                         | Returns the full tasks array.                                                                                                    |
| <code>getTaskById(id)</code>                       | Finds a task by its numeric ID using<br><code>Array.find</code> .                                                                |
| <code>addTask(title, description, priority)</code> | Instantiates a new<br><code>Task</code><br>and pushes it onto the array. Returns the new task.                                   |
| <code>updateTask(id, fields)</code>                | Looks up the task and calls<br><code>task.update(fields)</code><br>. Returns<br><code>null</code><br>if the task does not exist. |
| <code>deleteTask(id)</code>                        | Finds the task's index and removes it with<br><code>splice</code><br>. Returns<br><code>false</code><br>if not found.            |
| <code>getStats()</code>                            | Computes and returns total, completed, pending, and per-priority counts using<br><code>Array.filter</code> .                     |

Because the array lives in module scope and Node.js caches modules after the first

`require`

, every part of the application shares the same array instance throughout the lifetime of the process.

### `src/middleware/validate.js`

Exports two Express middleware functions used to validate request bodies before they reach route handlers.

`validateTask`

is applied to the

`POST /tasks`

route. It requires

`title`

and

`description`

to be present, non-empty strings. If

`priority`

is provided, it must be one of "low", "medium", or "high". If any rule fails, the middleware responds immediately with HTTP 400 and a JSON array of error messages, and the route handler is never called.

`validateTaskUpdate`

is applied to the

`PUT /tasks/:id`

route. The rules are the same as above, except all fields are optional. Only fields that are actually present in the request body are checked. This allows callers to send a partial update with only the fields they want to change.

Both functions call

`next()`

to hand control to the route handler when validation passes.

### `src/middleware/upload.js`

Configures and exports a Multer instance used for handling PDF file uploads.

Multer is configured with disk storage rather than memory storage, meaning uploaded files are written to the filesystem inside the

`uploads/`

directory rather than being held in a buffer. Each file is given a randomised filename composed of a millisecond timestamp and a large random integer, followed by the original file extension. This prevents filename collisions and avoids exposing the original filename on disk.

A file filter function inspects the MIME type of each incoming file. If it is not

`application/pdf`

, the filter rejects it with an error message, and Multer does not write the file. The size limit is set to 2 MB ( $2 \times 1024 \times 1024$  bytes).

### `src/routes/tasks.js`

Defines all ten API endpoints under the

`/tasks`

prefix using an Express

`Router`

. Each route delegates data operations to the store and returns a consistent JSON envelope.

| Method | Path            | Description                                                          |
|--------|-----------------|----------------------------------------------------------------------|
| GET    | /               | Returns all tasks. Supports completed and priority query parameters. |
| GET    | /stats          | Returns aggregate statistics.                                        |
| GET    | /:id            | Returns a single task by ID.                                         |
| POST   | /               | Creates a new task. Runs validateTask.                               |
| PUT    | /:id            | Updates a task. Runs validateTaskUpdate.                             |
| PATCH  | /:id/complete   | Marks a task as complete.                                            |
| DELETE | /:id            | Deletes a task.                                                      |
| POST   | /:id/attachment | Uploads a PDF and attaches metadata to the task object.              |

A route-level error handler is registered after all routes. It catches errors thrown by Multer (such as a file that exceeds 2MB or has the wrong MIME type) and converts them into structured JSON 400 responses, consistent with the rest of the API.

When a PDF is uploaded successfully, the route adds an

```
attachment
```

property directly to the task object containing the original filename, the randomised on-disk filename, the file size in bytes, and an upload timestamp.

## public/index.html

The single HTML page served to the browser. It defines the full layout of the UI:

- A header containing the application title and three live statistic cards showing total, completed, and pending task counts.
- A task creation form with fields for title, description, and priority.
- A filter bar with dropdowns for priority and completion status, and a button to clear all active filters.
- A task container where task cards are rendered dynamically.
- A modal overlay used for the task edit form.
- A toast notification element used for success and error messages.

The page loads

```
style.css
```

for styling and

```
app.js
```

for all interactivity. No frameworks or external CDN links are used.

## public/app.js

The browser-side JavaScript file. It manages all UI behaviour and communicates with the backend exclusively through the Fetch API using

```
async
```

```
/
```

```
await
```

```
.
```

On page load it fetches the current task list and statistics from the API and renders them. It then attaches event listeners to the form, filter dropdowns, clear button, and modal close button.

The file provides the following functional groups:

### API functions (

```
loadTasks
```

```
,
```

```
loadStats
```

```
,
```

```
handleAddTask
```

```
,
```

```
updateTask
```

```
,
```

```
completeTask
```

```
,
```

```
deleteTask
```

```
,
```

```
uploadAttachment
```

) — each makes a fetch call to the corresponding API endpoint, reads the JSON response, and either updates the UI or shows an error toast.

### Render functions (

```
renderTasks
```

```
,
```

```
showEditModal
```

```
,
```

```
closeModal
```

) —

```
renderTasks
```

maps the local task array to HTML strings and sets

```
innerHTML
```

on the task container.

```
showEditModal
```

builds an edit form inside the modal, pre-filled with the current task values, and attaches a submit handler that calls

```
updateTask
```

```
.
```

**Filter functions (**

```
handleFilterChange
```

```
,
```

```
clearFilters
```

) — update a local

```
filters
```

state object and re-fetch the task list with the appropriate query parameters appended to the URL.

**Utility functions (**

```
showToast
```

```
,
```

```
escapeHtml
```

```
,
```

```
formatBytes
```

) —

```
showToast
```

briefly displays a success or error message in the fixed toast element.

```
escapeHtml
```

sanitises user-supplied strings before inserting them into the DOM to prevent cross-site scripting.

```
formatBytes
```

formats a byte count into a human-readable string.

## public/style.css

Provides all visual styling for the frontend. Key design decisions include:

- CSS custom properties (variables) defined on

```
:root
```

for the colour palette, used consistently throughout the file.

- A CSS Grid layout for the task card area using

```
repeat(auto-fill, minmax(300px, 1fr))
```

, which automatically adjusts the number of columns based on available width.

- A fixed-position toast element that slides in from the bottom of the viewport using a CSS transition on

```
opacity
```

and

```
transform
```

.

- A modal that uses

```
position: fixed
```

and a blurred backdrop. It is hidden by default (

```
display: none
```

) and made visible by adding an

```
active
```

class.

- A responsive media query at 768px that switches the task grid to a single column and stacks the filter controls vertically.
- Colour-coded priority badges: green for low, amber for medium, and red for high.

### package.json

Declares the project as a CommonJS Node.js package (via

```
"type": "commonjs"
```

). Lists two production dependencies:

```
express
```

at version

```
^5.2.1
```

and

```
multer
```

at version

```
^2.1.1
```

. No development dependencies or test runner are configured.

### package-lock.json

The npm-generated lockfile. Records the exact resolved versions and integrity hashes for every package in the dependency tree, including transitive dependencies, ensuring reproducible installs across machines.

### test.http

A file containing 29 hand-written HTTP requests that cover every endpoint, including deliberate failure cases (missing fields, invalid priority values, non-PDF uploads, requests for non-existent task IDs). It is compatible with the JetBrains HTTP Client and the VS Code REST Client extension.

Requests that create tasks use response handler scripts to capture the returned task ID into a shared variable, which subsequent requests (update, complete, delete, upload) then reference

automatically. This makes it possible to run the full suite sequentially without manually copying IDs between requests.

### **.gitignore**

Instructs Git to exclude the following from version control:

- `node_modules/`  
— installed packages, reproducible via  
`npm install`  
.
- `uploads/`  
— user-uploaded PDF files, which are runtime artefacts and should not be committed.
- Log files (  
`*.log`  
,  
`npm-debug.log*`  
).
- Environment variable files (  
`.env`  
,  
`.env.local`  
).
- IDE configuration directories (  
`.idea/`  
).
- OS metadata files (  
`.DS_Store`  
,  
`Thumbs.db`  
).

### **.gitattributes**

Contains a single rule (  
`* text=auto`

) that instructs Git to automatically detect text files and normalise their line endings to LF on commit. This prevents line-ending inconsistencies when the project is worked on across Windows and Unix-based machines.

## API Summary

### Response envelopes

All responses use one of three JSON envelopes:

- `{"success": true, "data": {...}}`  
— successful response with a payload.
- `{"success": false, "message": "...}"`  
— single error message (404, runtime error).
- `{"success": false, "errors": ["..."]}`  
— validation error with one or more messages.

| Method | Endpoint              | Success Status |
|--------|-----------------------|----------------|
| GET    | /tasks                | 200            |
| GET    | /tasks?completed=true | 200            |
| GET    | /tasks?priority=high  | 200            |
| GET    | /tasks/stats          | 200            |
| GET    | /tasks/:id            | 200            |
| POST   | /tasks                | 201            |
| PUT    | /tasks/:id            | 200            |
| PATCH  | /tasks/:id/complete   | 200            |
| DELETE | /tasks/:id            | 200            |
| POST   | /tasks/:id/attachment | 200            |

## Design Notes and Limitations

**No persistence.** All task data lives in a JavaScript array in server memory. Restarting the server clears all tasks. Persistence could be added by replacing the store module with one that reads from and writes to a file or database, without changing any other part of the application.

**Single attachment per task.** Uploading a second PDF to a task overwrites the

`attachment`

metadata on the task object. The previously uploaded file is not deleted from disk automatically.

**IDs are timestamps.** Task IDs are generated using

`Date.now()`

, which returns the number of milliseconds since the Unix epoch. This is unique enough for a single-process, in-memory application but would not be suitable for a distributed or high-throughput system.

**No authentication.** All API endpoints are publicly accessible. Any client that can reach the server can read, create, modify, and delete any task.

**XSS mitigation.** The frontend escapes all user-supplied strings through a DOM-based HTML-escape function before inserting them into the page, preventing stored cross-site scripting via task titles or descriptions.